

Quais todos os registradores e suas funções para ARM

💡 Pensamento por 5 s

Os registradores do ARM variam conforme a arquitetura (ARMv7/AArch32 clássica vs. ARMv8/AArch64) e o perfil (A/R para aplicação, M para microcontroladores como Cortex-M). Abaixo está um resumo claro e completo das principais versões. developer.arm.com

1. ARM Clássico (ARMv7 / AArch32) – 32 bits

Registradores visíveis no modo User (16 registradores de 32 bits, R0–R15):

Registrador	Nome/Apelido	Função Principal
R0–R3	-	Uso geral (argumentos de função, resultados, temporários). R0 frequentemente armazena o retorno de funções.
R4–R11	-	Uso geral preservados (devem ser salvos/restaurados em chamadas de função). R11 frequentemente usado como Frame Pointer (FP) .
R12	IP (Intra-Procedure Call)	Scratch register, usado em chamadas intra-procedurais.
R13	SP (Stack Pointer)	Ponteiro de pilha (aponta para o topo da stack).
R14	LR (Link Register)	Armazena o endereço de retorno de uma sub-rotina (usado com BL).
R15	PC (Program Counter)	Contém o endereço da instrução atual (ou +8 devido ao pipeline em ARM state).

Registradores de Status:

- **CPSR** (Current Program Status Register): Flags de condição (**Negative**, **Zero**, **Carry**, **Overflow**), bits de modo do processador, estado Thumb, etc. [azercia-labs.com](https://www.azercia-labs.com)
- **SPSR** (Saved Program Status Register): Salva o CPSR ao entrar em modos de exceção (um por modo privilegiado).

Banked Registers (registradores "sombreados"): Em modos privilegiados (SVC, IRQ, FIQ, Abort, Undefined, System), alguns registradores são trocados (banked) para contexto rápido de interrupções. Exemplo: FIQ tem R8–R12 banked. Total pode chegar a ~37 registradores. [scribd.com](https://www.scribd.com)

2. ARMv8 AArch64 (64 bits) – Mais comum em processadores modernos (Cortex-A)

- **31 registradores de propósito geral de 64 bits: X0–X30** (podem ser acessados como 32 bits com W0–W30). developer.arm.com
 - **X0–X7:** Argumentos de função e retornos.
 - **X8:** Indirect result (para structs grandes).
 - **X9–X15:** Temporários (caller-saved).
 - **X16–X17:** Temporários intra-procedurais (IPO/IP1).
 - **X18:** Plataforma-specific (ex.: shadow stack em alguns SOs).
 - **X19–X28:** Preservados (callee-saved).
 - **X29: FP** (Frame Pointer).
 - **X30: LR** (Link Register / Procedure Link Register).
- **Registradores especiais:**
 - **SP** (Stack Pointer) — Pode ser SP_ELO, SP_EL1 etc. (por nível de exceção).
 - **PC** (Program Counter) — 64 bits.
 - **XZR / WZR:** Zero Register (leitura sempre retorna 0; escrita descartada).
 - **ELR_ELx** (Exception Link Register): Endereço de retorno de exceção por nível (ELO a EL3).
 - **SPSR_ELx:** Saved Program Status Register por nível.

- **Registradores SIMD/FP (Floating-Point/SIMD):** 32 registradores **V0–V31** (128 bits cada), acessíveis como Q, D, S, H, B.

Não há banking geral como no AArch32; os registradores são unbanked, mas SP, ELR e SPSR são por Exception Level (EL0 = user/app, EL1 = kernel, EL2 = hypervisor, EL3 = secure monitor). [embien.com](#)

3. Cortex-M (ARMv6-M / ARMv7-M / ARMv8-M) – Microcontroladores

Mais simples (sem banking complexo):

- **R0–R12:** Uso geral.
- **MSP** (Main Stack Pointer) e **PSP** (Process Stack Pointer).
- **LR, PC.**
- **PSR** (Program Status Register) — inclui APSR (Application PSR) com flags N/Z/C/V.
- **Registradores de sistema:** PRIMASK, BASEPRI, CONTROL, etc. (para interrupções e privilégios).

[developer.arm.com](#)

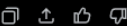
Resumo de Funções Comuns

- **Uso geral (R0–R12 / X0–X30):** Armazenar dados, endereços, cálculos.
- **SP:** Gerencia a pilha (push/pop, alocação local).
- **LR:** Retorno de funções/sub-rotinas.
- **PC:** Controle de fluxo (branching).
- **Registradores de Status (CPSR/PSR/SPSR):** Condições para execução condicional (ex.: BEQ, BNE) e estado do processador.

Convenções ABI (ex.: AAPCS): Definem como registradores são usados em chamadas de função (quais devem ser preservados, passagem de argumentos etc.).

Para detalhes oficiais, consulte a **ARM Architecture Reference Manual** (disponível no site da Arm Developer). A arquitetura exata depende do core (Cortex-A, R, M) e do modo de execução (AArch32 vs AArch64). Se precisar de foco em uma versão específica (ex.: Cortex-M3, ARMv8-A), avise!

 6 páginas da web



🔗 Explique convenções ABI AAPCS

🔗 Compare ARM com x86 registradores

e para ARM64

💭 Pensamento por 3 s

ARM64 (AArch64) – Registradores completos

A arquitetura **AArch64** (ARMv8 e superiores) é a versão de 64 bits do ARM. Ela difere bastante da versão de 32 bits (AArch32).

1. Registradores de Propósito Geral (GPR)

- **31 registradores** de 64 bits: **X0 a X30**.
- Podem ser acessados como **32 bits** usando **W0 a W30** (os 32 bits superiores são ignorados ou zerados dependendo da instrução).

Registrador	Nome/Apelido	Função (Convenção AAPCS64)
X0–X7	-	Argumentos de função (primeiros 8 parâmetros) e valores de retorno . Caller-saved (podem ser alterados pela função chamada).
X8	XR / Indirect Result	Usado para retornar structs grandes por referência (endereço do resultado).
X9–X15	Temporários	Scratch / caller-saved.

X16–X17	IPO / IP1	Registradores intra-procedurais (usados pelo linker para PLT, stubs etc.).
X18	Platform Register	Reservado para uso específico da plataforma (ex.: thread pointer em alguns SOs). Evite usar em código portátil.
X19–X28	-	Callee-saved (a função deve preservar o valor se usar).
X29	FP (Frame Pointer)	Ponteiro de frame da pilha (stack frame).
X30	LR (Link Register)	Endereço de retorno de sub-rotina (usado por BL).

- **X31**: Não é um registrador fixo. Depende do contexto:
 - Como **SP** (Stack Pointer) em instruções de pilha.
 - Como **XZR / WZR** (Zero Register) nas demais instruções (leitura = 0, escrita = descartada).

2. Registradores Especiais

- **SP** (Stack Pointer) → 64 bits. Pode ser **SP_ELO** (user) ou **SP_ELn** (níveis de exceção).
- **PC** (Program Counter) → 64 bits (não acessível diretamente como registrador na maioria das instruções).
- **NZCV** → Flags de condição (Negative, Zero, Carry, overflow). Equivalente ao CPSR do ARM32, mas mais simples.
- **ELR_ELx** (Exception Link Register) — Armazena o endereço de retorno ao sair de exceção/interrupção por nível (EL1, EL2, EL3).
- **SPSR_ELx** (Saved Program Status Register) — Salva o estado do processador ao entrar em exceção.

3. Registradores SIMD / Floating-Point / Vector

- **32 registradores de 128 bits: V0 a V31** (também chamados **Q0–Q31**).
- Acessos parciais:
 - **V / Q** → 128 bits
 - **D** → 64 bits (double)
 - **S** → 32 bits (single)
 - **H** → 16 bits
 - **B** → 8 bits

Usados para **NEON** (SIMD), ponto flutuante (FP) e criptografia.

SVE (Scalable Vector Extension) — opcional em alguns cores: registradores **Z0–Z31** (tamanho escalável) e predicados **P0–P15**.

4. Registradores de Sistema (System Registers)

Acessíveis apenas em modos privilegiados (EL1+). Exemplos importantes:

- **ID_AA64xxx_EL1** — Registradores de identificação de features da CPU.
- **TTBR0_EL1 / TTBR1_EL1** — Tabelas de tradução de memória (paginação).
- **CNTVCT_ELO** — Contador de timer.
- **TPIDR_ELO** — Thread ID register (usado por sistemas operacionais).

Existem dezenas de system registers por nível de exceção (ELO a EL3).

Resumo de Convenções (AAPCS64)

- **Parâmetros**: X0–X7 (inteiros/ponteiros) e V0–V7 (float/SIMD).
- **Retorno**: X0/X1 (inteiro) ou V0 (float).
- **Preservados**: X19–X28 + FP (X29) + LR (X30).
- A pilha deve ser mantida alinhada em **16 bytes**.

Observações:

- Não existe “banking” de registradores como no ARM32 (exceto SP e alguns system registers por EL1).

- Não existe "banking" de registradores como no ARMv32 (exceto SP e alguns system registers por EL).
- O processador tem Exception Levels: **EL0** (user), **EL1** (kernel), **EL2** (hypervisor), **EL3** (secure/monitor).

Se quiser um exemplo de código assembly, tabela completa de system registers, ou foco em algo específico (Cortex-A, SVE2, Neon, etc.), é só pedir!

Documentação oficial: **Arm Architecture Reference Manual** (DDI 0487) e **AAPCS64**.



↪ Exemplos de código assembly ARM64

↪ Diferenças entre AArch32 e AArch64

 Fazer uma pergunta

A new version of X is available. **Atualizar**

⚡ Rápido ▾

